

```

ubuntu@zabbix-server:~$ sudo systemctl start zabbix-server
ubuntu@zabbix-server:~$ sudo systemctl status zabbix-server --no-pager
● zabbix-server.service - Zabbix Server
   Loaded: loaded (/usr/lib/systemd/system/zabbix-server.service; disabled; preset: enabled)
   Active: active (running) since Sun 2026-09-27 07:51:03 UTC; 7s ago
   Process: 42615 ExecStart=/usr/sbin/zabbix_server -c $CONFFILE (code=exited, status=0/SUCCESS)
   Main PID: 42617 (zabbix_server)
     Tasks: 77 (limit: 2209)
    Memory: 60.8M (peak: 61.1M)
       CPU: 463ms
    CGroup: /system.slice/zabbix-server.service
            └─42617 /usr/sbin/zabbix_server -c /etc/zabbix/zabbix_server.conf
              └─42619 "/usr/sbin/zabbix_server: ha manager"
                └─42620 "/usr/sbin/zabbix_server: service manager #1 [processed 0 ev...
                  └─42621 "/usr/sbin/zabbix_server: configuration syncer [synced confi...
                    └─42625 "/usr/sbin/zabbix_server: alert manager #1 [sent 0, failed 0...
                      └─42626 "/usr/sbin/zabbix_server: alerter #1 started"

```

Live-lab evidence: after the database was initialized, zabbix-server reported Active: active (running).

6. Configure Nginx and the Zabbix frontend

The package installs the Zabbix Nginx configuration. For this classroom lab, expose the frontend on standard HTTP port 80 so students can open `http://`.

6.1 Check the packaged Nginx configuration

Inspect before editing

```
sudo grep -HE 'listen|server_name|root' /etc/zabbix/nginx.conf
```

During the live build, the packaged configuration was initially listening on 8080. That caused confusion because the trainer expected port 80. For this lab, change the Zabbix Nginx server block to listen on 80.

Edit the Zabbix Nginx config

```
sudo nano /etc/zabbix/nginx.conf
```

Use the package's existing configuration and change the listening port from:

Change

```
listen 8080,
```

to:

Final classroom setting

```
listen 80,
```

Do not replace the entire file with a hand-written config unless you have a specific reason; keeping the package-provided PHP/frontend settings reduces configuration errors.

6.2 Remove the default Nginx site if it conflicts with port 80

Run if the default site is enabled

```
sudo rm -f /etc/nginx/sites-enabled/default
sudo nginx -t
sudo systemctl restart nginx
sudo systemctl enable nginx
sudo systemctl status nginx --no-pager
```

6.3 Verify what is actually listening

Use a precise port check

```
sudo ss -tnpt | grep -HE ':80\s|:10051\s'
```

Avoid `grep ':80'` by itself: it also matches 8080. This exact ambiguity occurred during the live build.

6.4 Local HTTP test

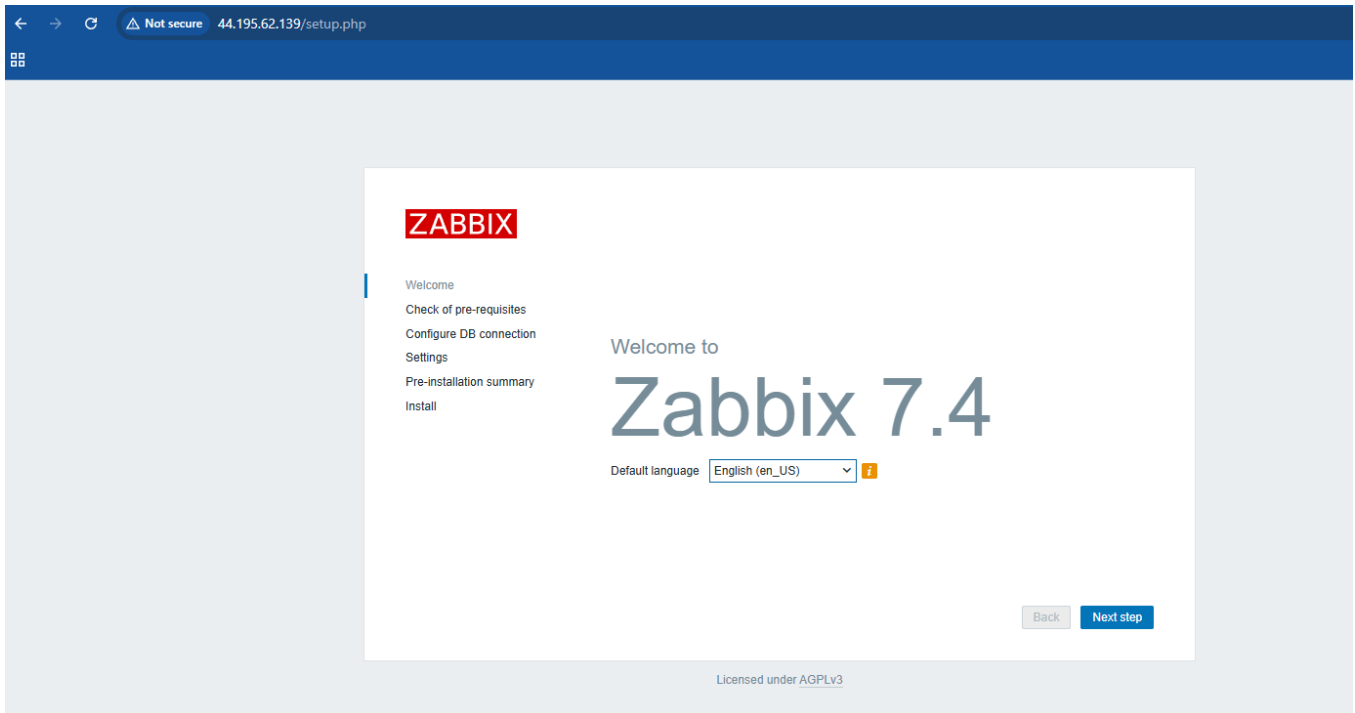
Test from the server itself

```
curl -I http://127.0.0.1
```

A healthy initial response may be HTTP/1.1 302 Found with Location: `setup.php`. That means the frontend is reachable and is redirecting to the installation wizard.

7. Complete the Zabbix Web Installation Wizard

Open `http://SERVER_PUBLIC_IP/` from the trainer workstation. The first page should show the Zabbix 7.4 welcome screen.

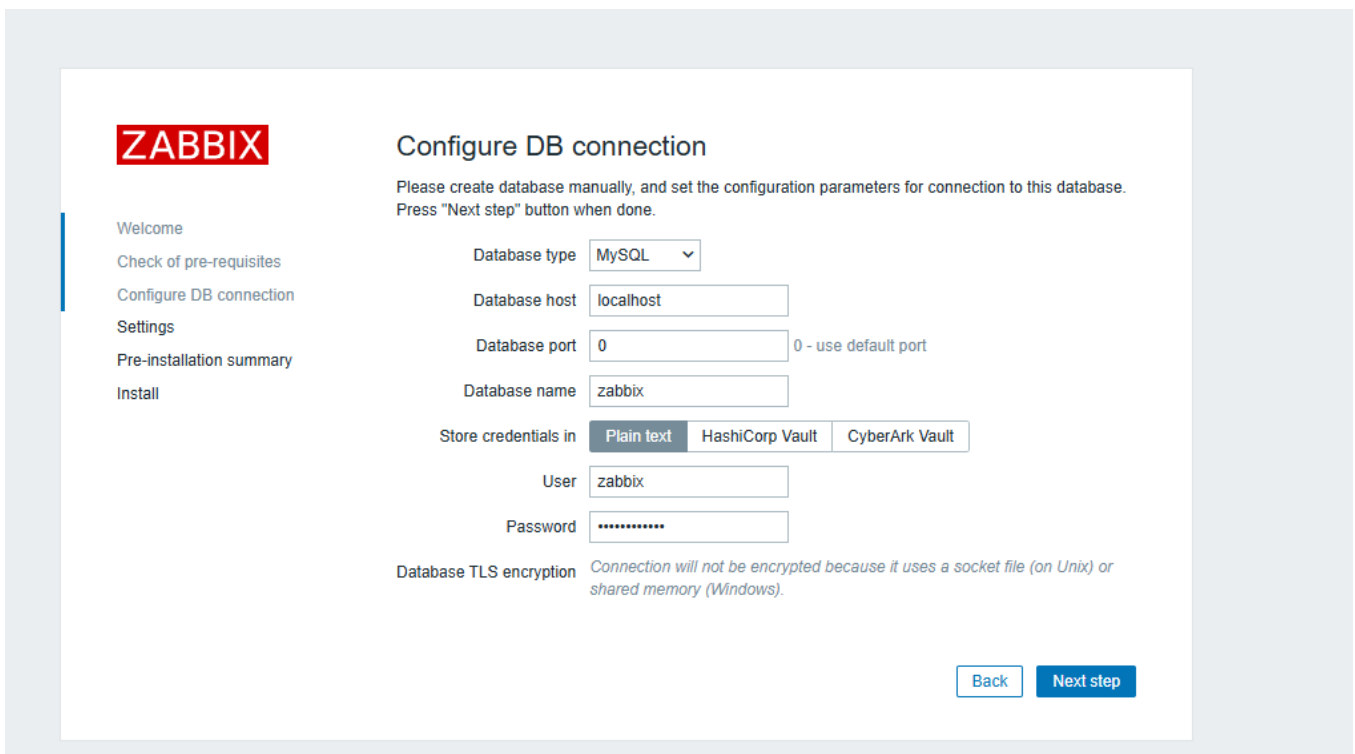


Live-lab evidence: Zabbix 7.4 web installation wizard.

7.1 Pre-requisite page

Continue through the prerequisite checks. If PHP extensions are missing, install the package named by the wizard and restart PHP-FPM/Nginx.

7.2 Database connection

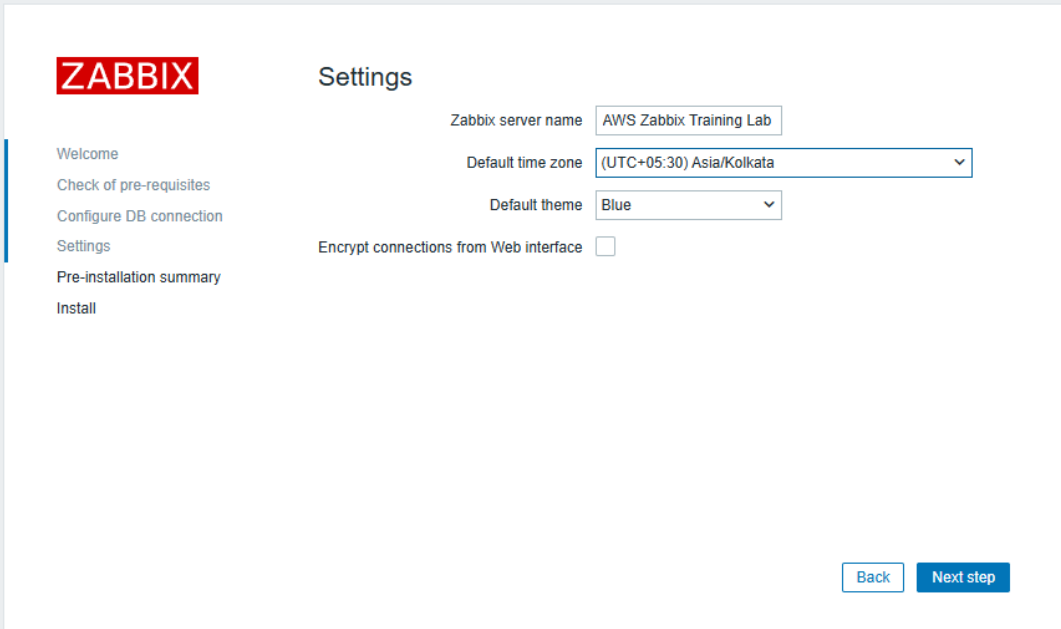


Live-lab evidence: MySQL database connection screen.

Field	Value
Database type	MySQL
Database host	localhost
Database port	0 / default
Database name	zabbix
User	zabbix
Password	The password created in Section 4
Database TLS	false for this single-EC2 classroom lab

The wizard may display a note that the database connection is not encrypted because it uses a local Unix socket. That is acceptable for this disposable single-host lab; production designs should follow your organization's database security requirements.

7.3 Settings



ZABBIX Settings

Welcome
Check of pre-requisites
Configure DB connection
Settings
Pre-installation summary
Install

Zabbix server name: AWS Zabbix Training Lab

Default time zone: (UTC+05:30) Asia/Kolkata

Default theme: Blue

Encrypt connections from Web interface: ☐

Back Next step

Licensed under AGPLv3

Live-lab evidence: Settings page with the classroom server name and Asia/Kolkata timezone.

Recommended classroom values:

Setting	Value
Zabbix server name	AWS Zabbix Training Lab
Default time zone	(UTC+05:30) Asia/Kolkata
Default theme	Blue
Encrypt web-interface connections	Leave off for this initial HTTP-only lab

7.4 Pre-installation summary



Pre-installation summary

Please check configuration parameters. If all is correct, press "Next step" button, or "Back" button to change configuration parameters.

Welcome

Check of pre-requisites

Configure DB connection

Settings

Pre-installation summary

Install

Database type **MySQL**

Database server **localhost**

Database port **default**

Database name **zabbix**

Database user **zabbix**

Database password *********

Database TLS encryption **false**

Zabbix server name **AWS Zabbix Training Lab**

Encrypt connections from Web interface **false**

Back

Next step

Licensed under AGPL v3

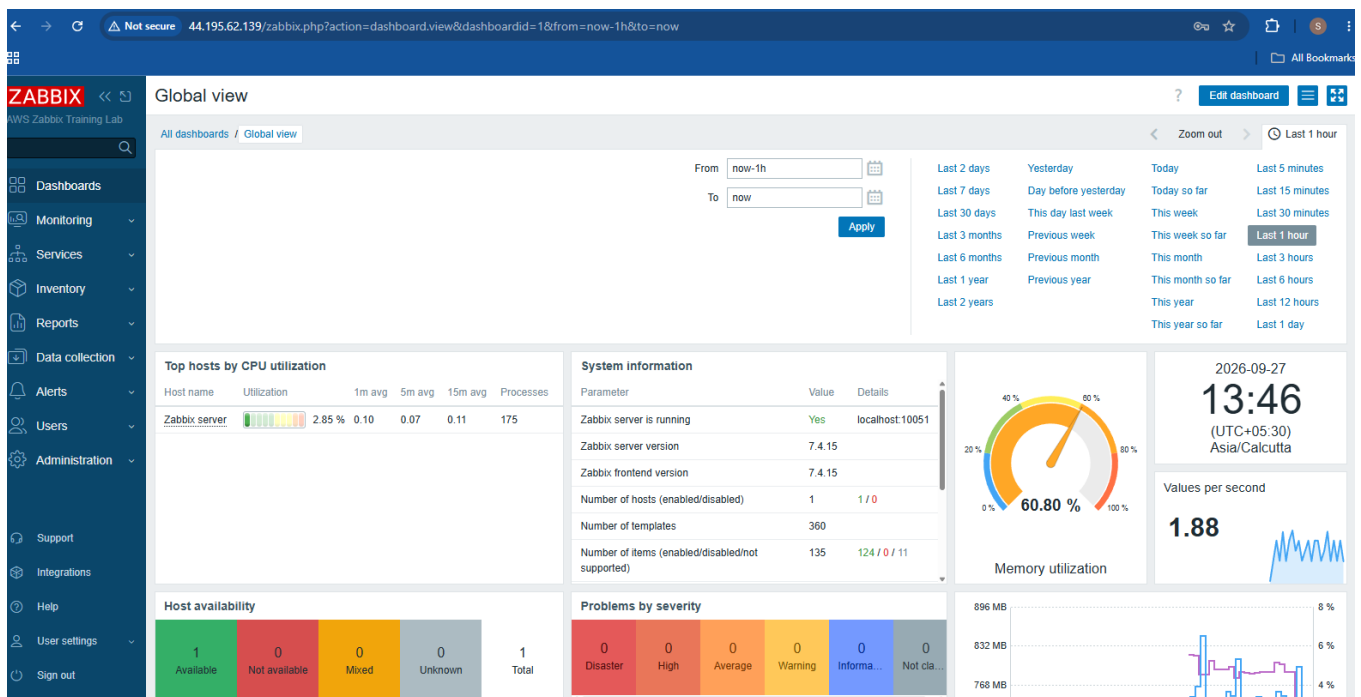
Live-lab evidence: final pre-installation summary.

8. First login and baseline validation

Complete the wizard and sign in using the default administrative account created by the Zabbix installation wizard. Change the password immediately in any reusable or shared environment.

Baseline checks

- Dashboard shows Zabbix server running.
- Zabbix server and frontend versions are visible.
- The server host is available.
- CPU and memory graphs begin populating.
- No database initialization errors remain.



Live-lab evidence: Zabbix Global view after successful installation. The dashboard showed Zabbix Server running and version 7.4.15.

Teaching point: the server itself can be monitored as a host. The dashboard is not the monitoring data source; the Zabbix server collects/stores data and the frontend visualizes it.

9. Build the monitored Linux EC2

Create a second Ubuntu 24.04 EC2 named zabbix-client. Put it in the same VPC as the server. Record its private IP; the live lab client was 172.31.13.22, but your address will differ.

9.1 Install Agent 2

Run on zabbix-client

```
sudo apt update
sudo apt upgrade -y

wget https://repo.zabbix.com/zabbix/7.4/release/ubuntu/pool/main/z/zabbix-release/zabbix-release_latest+ubuntu24.04_all.deb
sudo dpkg -i zabbix-release_latest+ubuntu24.04_all.deb
sudo apt update

sudo apt install -y zabbix-agent2
```

Zabbix Agent 2 is installed on the monitored host and can be extended through plugins. The official documentation recommends the package method and then linking the appropriate template to the host.

[\[cite\]turn0search10\[turn0search1\]](#)

9.2 Configure the agent

Edit

```
sudo nano /etc/zabbix/zabbix_agent2.conf
```

Set these values:

Passive-agent configuration used by this lab

```
Server=<ZABBIX_SERVER_PRIVATE_IP>
Hostname=zabbix-client
```

Use the private IP of the Zabbix Server for Server=. Do not put the public IP of the server here for this VPC lab.

9.3 Start and validate Agent 2

Run on zabbix-client

```
sudo systemctl restart zabbix-agent2
sudo systemctl enable zabbix-agent2
sudo systemctl status zabbix-agent2 --no-pager
```

```
ubuntu@ip-172-31-13-22:~$ sudo systemctl status zabbix-agent2 --no-pager
● zabbix-agent2.service - Zabbix Agent 2
   Loaded: loaded (/usr/lib/systemd/system/zabbix-agent2.service; enabled; preset: enabled)
   Active: active (running) since Sun 2026-09-27 08:32:12 UTC; 31s ago
     Main PID: 29244 (zabbix_agent2)
        Tasks: 7 (limit: 1013)
      Memory: 3.8M (peak: 4.2M)
         CPU: 43ms
       CGroup: /system.slice/zabbix-agent2.service
               └─29244 /usr/sbin/zabbix_agent2 -c /etc/zabbix/zabbix_agent2.conf
```

```
Sep 27 08:32:11 zabbix-client systemd[1]: Starting zabbix-agent2.service - Zabbix Agent 2...
Sep 27 08:32:11 zabbix-client zabbix_agent2[29238]: Validating configuration file "/etc/zabbi...onf"
Sep 27 08:32:12 zabbix-client zabbix_agent2[29238]: Validation successful
Sep 27 08:32:12 zabbix-client systemd[1]: Started zabbix-agent2.service - Zabbix Agent 2.
Sep 27 08:32:12 zabbix-client zabbix_agent2[29244]: Starting Zabbix Agent 2 (7.4.15)
Sep 27 08:32:12 zabbix-client zabbix_agent2[29244]: Zabbix Agent2 hostname: [zabbix-client]
Sep 27 08:32:12 zabbix-client zabbix_agent2[29244]: Press Ctrl+C to exit.
Hint: Some lines were ellipsized, use -l to show in full.
ubuntu@ip-172-31-13-22:~$
```

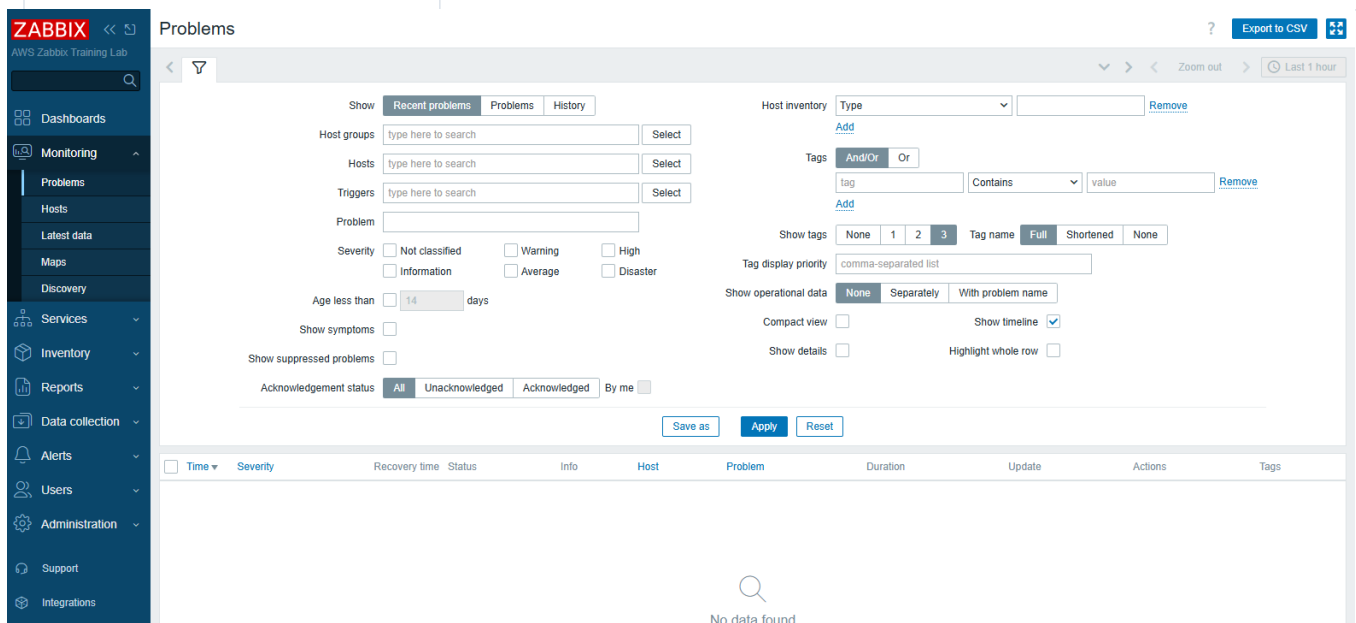
Live-lab evidence: zabbix-agent2 active (running), with hostname Zabbix-client.

10. Add zabbix-client to Zabbix

In the web UI go to Data collection → Hosts → Create host.

Host configuration

Field	Value
Host name	Zabbix-client
Template	Linux by Zabbix agent
Host group	Linux servers
Interface type	Agent
IP address	Client private IP, e.g. 172.31.13.22
Port	10050
Monitored by	Server
Enabled	Yes



Live-lab evidence: host configuration screen used for Zabbix-client.

Why these fields matter

- Host identifies the monitored endpoint.
- Interface tells Zabbix where the Agent can be reached.
- Template supplies reusable monitoring definitions.
- Host group organizes hosts for administration and filtering.
- Port 10050 is the default Zabbix Agent port.

After adding the host, wait for the Agent availability icon to become green. If it stays red, troubleshoot the Security Group, private IP, agent hostname and agent service before changing templates.

ZABBIX << >> AWS Zabbix Training Lab

Hosts

Host Wizard Create host Import

Filter

Host groups: type here to search Select

Templates: type here to search Select

Name:

DNS:

IP:

Port:

Status: Any Enabled Disabled

Monitored by: Any Server Proxy Proxy group

Tags: And/Or Or

tag: Contains value: Remove

Add

Apply Reset

☐	Name ▲	Items	Triggers	Graphs	Discovery	Web	Interface	Proxy	Templates	Status	Availability	Agent encryption	Info	Tags
☐	+++ Zabbix-client	Items 75	Triggers 30	Graphs 16	Discovery 3	Web	172.31.13.22:10050		Linux by Zabbix agent	Enabled	ZBX	None		
☐	+++ Zabbix server	Items 153	Triggers 83	Graphs 16	Discovery 6	Web	127.0.0.1:10050		Linux by Zabbix agent, Zabbix server health	Enabled	ZBX	None		

Displaying 2 of 2 found

0 selected Enable Disable Export Mass update Delete

Zabbix 7.4.15. © 2001–2026, Zabbix SIA

Live-lab evidence: both Zabbix-client and Zabbix server were green/available; the client had 75 items, 30 triggers and 16 graphs.

11. Explain Host → Template → Item → Trigger

This is the conceptual core of the class. Zabbix documentation defines items as individual metrics and triggers as logical expressions that evaluate item data. Templates package reusable entities so they can be applied to hosts. [cite]turn0search6[turn0search3][turn0search4]

Object	Question it answers	Example in this lab
Host	What machine are we monitoring?	Zabbix-client
Template	What monitoring definitions should be applied to this host?	Linux by Zabbix agent
Item	What data should be collected?	system.cpu.util
Trigger	When should the data be treated as a problem?	tr{system.cpu.util[,user]} > 90%
Problem/Event	What incident state exists now?	High CPU utilization
Action/Notification	Who/what should be informed?	Not configured in this base demo

11.1 Inspect the items

Name	Triggers	Key	Interval	History	Trends	Type	Status	Tags	Info
Linux by Zabbix agent: Available memory	Triggers 1	vm.memory.size[available]	1m	31d	365d	Zabbix agent	Enabled	component: memory	
Linux by Zabbix agent: Available memory in %		vm.memory.size[pavailable]	1m	31d	365d	Zabbix agent	Enabled	component: memory	
Linux by Zabbix agent: Checksum of /etc/passwd	Triggers 1	vfs.file.cksum[/etc/passwd.sha256]	15m	31d		Zabbix agent	Enabled	component: security	
Linux by Zabbix agent: Context switches per second		system.cpu.switches	1m	31d	365d	Zabbix agent	Enabled	component: cpu	
Linux by Zabbix agent: CPU guest nice time		system.cpu.util[,guest_nice]	1m	31d	365d	Zabbix agent	Enabled	component: cpu	
Linux by Zabbix agent: CPU guest time		system.cpu.util[,guest]	1m	31d	365d	Zabbix agent	Enabled	component: cpu	
Linux by Zabbix agent: CPU idle time		system.cpu.util[,idle]	1m	31d	365d	Zabbix agent	Enabled	component: cpu	
Linux by Zabbix agent: CPU interrupt time		system.cpu.util[,interrupt]	1m	31d	365d	Zabbix agent	Enabled	component: cpu	
Linux by Zabbix agent: CPU iowait time		system.cpu.util[,iowait]	1m	31d	365d	Zabbix agent	Enabled	component: cpu	
Linux by Zabbix agent: CPU nice time		system.cpu.util[,nice]	1m	31d	365d	Zabbix agent	Enabled	component: cpu	
Linux by Zabbix agent: CPU softirq time		system.cpu.util[,softirq]	1m	31d	365d	Zabbix agent	Enabled	component: cpu	
Linux by Zabbix agent: CPU steal time		system.cpu.util[,steal]	1m	31d	365d	Zabbix agent	Enabled	component: cpu	
Linux by Zabbix agent: CPU system time		system.cpu.util[,system]	1m	31d	365d	Zabbix agent	Enabled	component: cpu	
Linux by Zabbix agent: CPU user time		system.cpu.util[,user]	1m	31d	365d	Zabbix agent	Enabled	component: cpu	
Linux by Zabbix agent: CPU idle time: CPU utilization	Triggers 1	system.cpu.util		31d	365d	Dependent item	Enabled	component: cpu	
Linux by Zabbix agent: Free swap space		system.swap.size[,free]	1m	31d	365d	Zabbix agent	Enabled	component: memory component: storage	
Linux by Zabbix agent: Free swap space in %	Triggers 1	system.swap.size[,pfree]	1m	31d	365d	Zabbix agent	Enabled	component: memory component: storage	
Mounted filesystem discovery: Get filesystems: FS [/boot]: Get data		vfs.fs.dependent[/boot,data]	1h			Dependent item	Enabled	component: raw component: storage filesystem: /boot ***	
Mounted filesystem discovery: FS [/boot]: Get data: FS [/boot]: Inodes: Free in %	Triggers 2	vfs.fs.dependent.inode[/boot,pfree]	31d	365d		Dependent item	Enabled	component: storage filesystem: /boot fstype: ext4	

Live-lab evidence: the Linux template supplied CPU, memory, storage and other items; the host had 75 items.

Notice examples such as system.cpu.util[,user], system.cpu.util[,system], system.cpu.util[,idle] and dependent/calculated items. This is a useful opportunity to explain that a template can contain many related metrics and that not every displayed metric requires an independent manual configuration.

11.2 Inspect the CPU trigger

ZABBIX << >> AWS Zabbix Training Lab Dashboards Monitoring Services Inventory Reports Data collection Template groups Host groups Templates Hosts Maintenance Event correlation Discovery Alerts Users Administration Support	☐	Warning	OK	Mounted filesystem discovery: Linux: FS [/]: Running out of free inodes Depends on: Zabbix-client: Linux: FS [/]: Running out of free inodes	Free inodes: (ITEM.LASTVALUE1)	$\min((\text{Zabbix-client/vfs.fs.dependent.inode}/\text{pfree}), 5m) < (\$VFS.FS.INODE.PFREE.MIN.WARN: "7")$	Enabled	scope: capacity scope: performance
	☐	Average	OK	Mounted filesystem discovery: Linux: FS [/]: Space is critically low	Space used: ((ITEM.LASTVALUE1).freetotal(1))%	$\min((\text{Zabbix-client/vfs.fs.dependent.size}/\text{pused}), 5m) < (\$VFS.FS.PUSED.MAX.CRIT: "7")$	Enabled	scope: availability scope: capacity
	☐	Warning	OK	Mounted filesystem discovery: Linux: FS [/]: Space is low Depends on: Zabbix-client: Linux: FS [/]: Space is critically low	Space used: ((ITEM.LASTVALUE1).freetotal(1))%	$\min((\text{Zabbix-client/vfs.fs.dependent.size}/\text{pused}), 5m) < (\$VFS.FS.PUSED.MAX.WARN: "7")$	Enabled	scope: availability scope: capacity
	☐	Warning	OK	Linux by Zabbix agent: Linux: Getting closer to process limit	(ITEM.LASTVALUE1) active, (ITEM.LASTVALUE2) limit	$\text{last}((\text{Zabbix-client/proc.num})/\text{last}((\text{Zabbix-client/kernel.maxproc}) * 100 > 80$	Enabled	scope: performance
	☐	Warning	OK	Linux by Zabbix agent: Linux: High CPU utilization Depends on: Zabbix-client: Linux: Load average is too high	Current utilization: (ITEM.LASTVALUE1)	$\min((\text{Zabbix-client/system.cpu.util}, 5m) > (\$CPU.UTIL.CRIT)$	Enabled	scope: performance
	☐	Average	OK	Linux by Zabbix agent: Linux: High memory utilization Depends on: Zabbix-client: Linux: Lack of available memory		$\min((\text{Zabbix-client/vm.memory.util}, 5m) > (\$MEMORY.UTIL.MAX)$	Enabled	scope: capacity scope: performance
	☐	Warning	OK	Linux by Zabbix agent: Linux: High swap space usage Depends on: Zabbix-client: Linux: High memory utilization Zabbix-client: Linux: Lack of available memory	Free: (ITEM.LASTVALUE1), total: (ITEM.LASTVALUE2)	$\max((\text{Zabbix-client/system.swap.size}[\text{pfree}], 5m) < (\$SWAP.PFREE.MIN.WARN) \text{ and } \text{last}((\text{Zabbix-client/system.swap.size}[\text{total}]) > 0$	Enabled	scope: capacity
	☐	Information	OK	Network interface discovery: Linux: Interface ens5: Ethernet has changed	Current reported speed: (ITEM.LASTVALUE1)	Problem: $\text{change}((\text{Zabbix-client/vfs.file.contents}["\text{sys/class/net/ens5/speed}"]) < 0 \text{ and } \text{last}((\text{Zabbix-client/vfs.file.contents}["\text{sys/class/net/ens5/speed}"]) > 0 \text{ and } \text{last}((\text{Zabbix-client/vfs.file.contents}["\text{sys/class/net/ens5/speed}"]) > 0$	Unknown	scope: performance

Live-lab evidence: Linux: High CPU utilization trigger with expression
 $\min((\text{Zabbix-client/system.cpu.util}, 5m) > \{ \$CPU.UTIL.CRIT \})$.

In this template, the trigger is effectively minimum CPU utilization over the last 5 minutes > the critical macro value (90% in this lab). This explains why a brief 100% spike did not immediately create a problem.

12. The live CPU incident demonstration

This is the classroom moment to demonstrate the complete observability lifecycle.

12.1 Start with normal CPU

On zabbix-client

```
top  
# observe CPU  
# press q to exit
```

12.2 Generate CPU load

Start one CPU worker

```
yes > /dev/null &
```

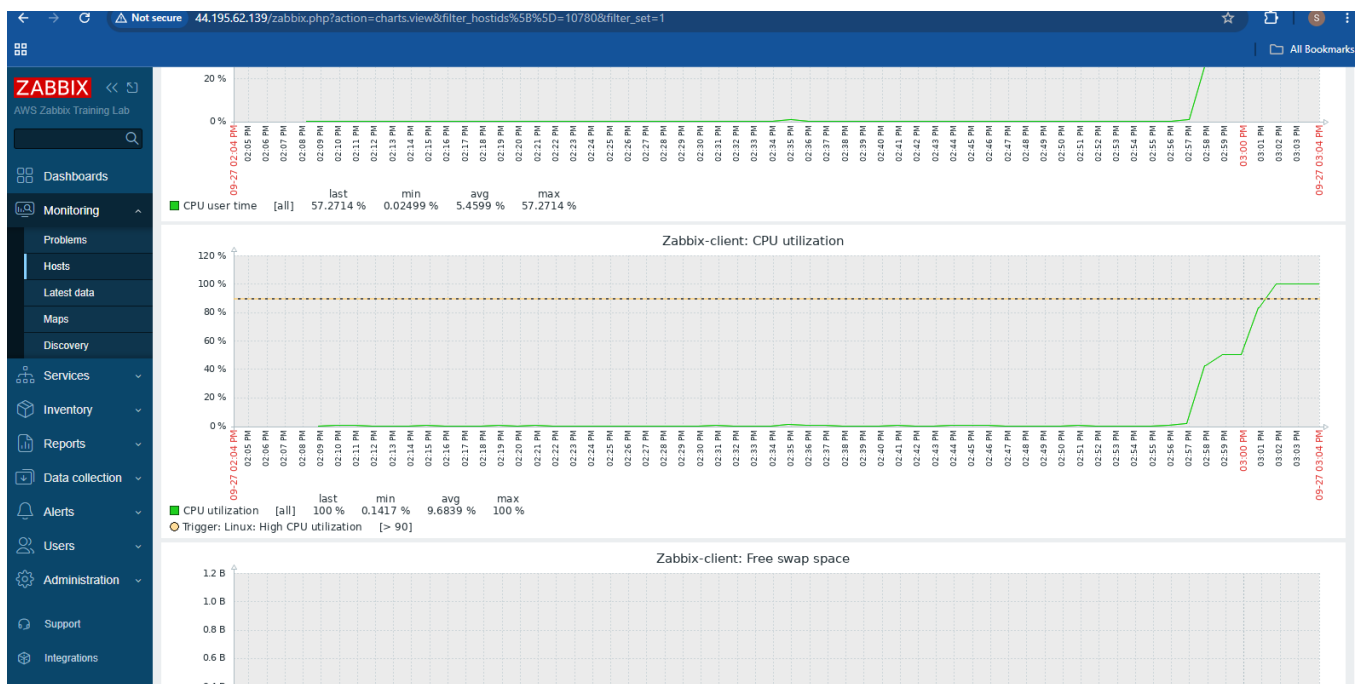
Because t3.micro has limited CPU capacity, one busy process may produce roughly half of total CPU capacity. For a stronger demonstration, start a second worker:

Second worker

```
yes > /dev/null &
```

12.3 Watch Zabbix

Go to Monitoring → Hosts → Zabbix-client and open the CPU utilization graph. The live run reached 100%.



Live-lab evidence: CPU utilization climbed to 100%; the trigger line is at 90%.

12.4 Explain why the alert is not immediate

The trigger uses $\min(\dots, 5m) > 90$, not simply the latest CPU value. The rolling five-minute minimum must be above 90%. This is a good teaching example of alert noise reduction: a short spike can be visible as a metric without immediately becoming an incident.

12.5 Wait for the trigger to fire

Keep both CPU workers running until the rolling five-minute condition is satisfied. The event should become a Warning/PROBLEM with wording similar to Linux: High CPU utilization (over 90% for 5m).

13. Problem, acknowledgement and recovery

Once the trigger fires, go to Monitoring → Problems.

The screenshot shows the Zabbix web interface with the 'Problems' tab selected. The left sidebar contains navigation links for Dashboards, Monitoring (Problems, Hosts, Latest data, Maps, Discovery), Services, Inventory, Reports, Data collection, Alerts, Users, Administration, Support, and Integrations. The main content area displays filters for Host groups, Hosts, Triggers, Problem, Severity, Age less than, Show symptoms, Show suppressed problems, and Acknowledgement status. A table lists the active problem: 'Linux: High CPU utilization (over 90% for 5m)' with a severity of 'Warning' and status of 'PROBLEM'. The table columns include Time, Severity, Recovery time, Status, Info, Host, Problem, Duration, Update, Actions, and Tags. The problem is associated with the host 'Zabbix-client' and has a duration of '1m 4s'. The tags are 'class: os', 'component: cpu', and 'scope: performance'.

Live-lab evidence: the active Warning/PROBLEM event on Zabbix-client.

13.1 Explain the event

- Host: Zabbix-client.
- Problem: Linux: High CPU utilization (over 90% for 5m).
- Severity: Warning.
- Status: PROBLEM.
- Duration: how long the problem has remained active.

13.2 Optional acknowledgement demonstration

Open the event and use the update/acknowledgement function. Explain that acknowledgement records that an operator has seen the event; it does not by itself resolve the underlying problem.

13.3 Recover the system

Stop the CPU workers

kill yes

Check the client with top and then return to Zabbix. After new data arrives and the trigger condition becomes false, the event should recover. Use Monitoring → Problems → History to show the lifecycle if the recovered event is no longer visible in the active-problem view.

Zabbix trigger documentation describes OK and Problem states and explains that expressions are recalculated as new values arrive; a simple problem expression also acts as the recovery threshold unless a separate recovery expression is defined. [cite]turn0search3[turn0search32]

14. Troubleshooting guide — problems we actually encountered

Symptom	Root cause / check	Corrective action
zabbix-server repeatedly restarts	Database exists but Zabbix schema was not imported	Install zabbix-sql-scripts; locate server.sql.gz; import
Table 'zabbix.users' doesn't exist	Initial database schema is missing.	Run the zcat/mysql import from Section 4.3.
server.sql.gz path not found	Wrong path was assumed.	Use dpkg -L zabbix-sql-scripts to locate the actual path.
Nginx browser connection refused	Nginx was listening on 8080 while the browser was setting Zabbix 80	Set the Zabbix Nginx server block to listen 80; restart Nginx
ss grep ':80' appears to show 8080	The grep pattern also matches 8080.	Use grep -E ':80\s :10051\s'.
curl 127.0.0.1:80 fails	Nothing is listening on port 80 or the wrong Nginx server block is enabled	Check Nginx enabled zabbix/nginx.conf and site
Zabbix client availability is red	Agent, private IP, hostname or Security Group issues	Check agent status, Server= value, Hostname= value
CPU graph reaches 100% but no trigger	Trigger is min(CPU,5m)>90, so the rolling five-minute average must be above 90%.	Keep on hand a utility that can read the Zabbix database and calculate the rolling average for the last 5 minutes.
CPU demo only reaches ~40-50%	Only one process consuming about one CPU core	Search the server logs for the process that is consuming the CPU

Server log commands

Use these first

```
sudo systemctl status zabbix-server --no-pager
sudo tail -50 /var/log/zabbix/zabbix_server.log
```

Agent log/service commands

On zabbix-client

```
sudo systemctl status zabbix-agent2 --no-pager
sudo journalctl -u zabbix-agent2 -n 50 --no-pager
```

15. Final pre-class verification checklist

Check	Command / UI	Expected
Zabbix server	systemctl status zabbix-server	active (running)
Nginx	systemctl status nginx	active (running)
HTTP	curl -I http://127.0.0.1	302 to setup.php initially, then frontend response at
Zabbix port	ss -lntp grep 10051	server listening on 10051
Agent client	systemctl status zabbix-agent2	active (running)
Client port	ss -lntp grep 10050	agent listening on 10050
Zabbix UI	Data collection → Hosts	Zabbix-client green
Items	Zabbix-client → Items	Linux metrics present
Triggers	Zabbix-client → Triggers	High CPU trigger enabled
Incident	Monitoring → Problems	Problem appears after sustained CPU load

Trainer tip

Run the incident demo once before class and leave the Zabbix UI open on the host graph. During class, have students predict what should happen before you start the CPU load. Then make them explain why the graph can reach 100% before the Problem appears. This turns the demo into an observability lesson rather than a button-click exercise.

16. Reset, cleanup and optional extensions

Reset the CPU demonstration

On the client

pkitt yes

AWS cost hygiene

- Stop both EC2 instances when the lab is not in use.
- Delete unused Elastic IPs if any are allocated.
- Remove the lab security groups, volumes and snapshots when the course is finished if they are no longer needed.
- For repeated classes, consider an AMI or automated build rather than manually rebuilding every time.

Recommended next Zabbix modules

Module	Classroom extension
Notifications	Configure email or another channel so a PROBLEM generates a notification.
Custom trigger	Create a trainer-owned trigger such as sustained CPU >70% for 2 minutes.
Nginx monitoring	Install/use the Nginx template and demonstrate web-server metrics.
Filesystem incident	Fill disk space in a controlled way and demonstrate a storage trigger.
Discovery	Show filesystem/network discovery creating monitoring entities automatically.
Dashboards	Build a trainer dashboard for CPU, memory, disk, availability and Problems.
Tags	Use tags to filter and route events by environment, service and component. [cite]turn0search15
Active checks	Compare passive checks used in this lab with Agent 2 active checks.

17. Official references

The runbook was validated against the Zabbix 7.4 documentation and the live classroom build. The official documentation should be used for version-specific changes before repeating the lab on a future release.

Reference	Use
Zabbix 7.4 Installation and first steps	Overall installation sequence and components. [cite\turn1search0]
Zabbix 7.4 Installation from packages	Official package repository guidance. [cite\turn1search1]
Zabbix 7.4 Requirements	Supported database, web server, PHP and Agent 2 requirements. [cite\turn1search2]
Zabbix 7.4 Items	Item/metric and key concepts. [cite\turn0search6\turn0search12]
Zabbix 7.4 Triggers	Trigger evaluation, Problem/OK states and time-window functions. [cite\turn0search3]
Zabbix 7.4 Templates	Reusable monitoring definitions and template inheritance. [cite\turn0search4\turn0search5]
Zabbix Agent 2	Agent 2 operation and package-based installation. [cite\turn0search10]

End of runbook

The final demonstrated lifecycle is: Host → Template → Item → Metric → Trigger → Problem → Acknowledge → Recovery.